

# RISC-V Interrupt Handling — Exam Notes

## 1. What is an Interrupt?

An interrupt is a hardware or software event that temporarily stops the normal execution of a program so the processor can handle an urgent event. After servicing the interrupt, execution resumes from where it was interrupted.

## 2. Why Interrupts are Needed

Interrupts improve CPU utilization by eliminating constant polling of devices. Devices notify the CPU only when service is required.

## 3. PLIC and CLINT

PLIC (Platform-Level Interrupt Controller) handles external device interrupts.

CLINT (Core Local Interruptor) handles timer and software interrupts.

## 4. Important Registers

mip – Pending interrupt register.

mie – Enables/disables specific interrupt sources.

mstatus – Processor status register containing MIE bit.

mtvec – Address of ISR.

mepc – Return address after interrupt servicing.

mcause – Stores interrupt/exception cause.

## 5. Interrupt Service Routine (ISR)

Special software routine executed when an interrupt occurs. It identifies the source, performs service, clears the interrupt condition, and returns control to the interrupted program.

## 6. Interrupt Handling Sequence

### Phase A: Hardware Response

1. Finish current instruction.
2. Save next instruction address in mepc.
3. Save processor state in mstatus.
4. Disable interrupts (MIE = 0).
5. Jump to ISR address stored in mtvec.

### Phase B: Software Response

1. Save registers to stack.
2. Service interrupt.
3. Restore registers.

### Phase C: Return

ISR ends with mret.

mret restores interrupt state, loads PC from mepc, and resumes execution.

## 7. Why Manual Register Saving?

RISC-V follows RISC principles and keeps hardware simple. Register saving is handled by software for flexibility and simpler hardware.

## 8. Intel vs RISC-V

Intel uses INTR pin and IRET instruction.

RISC-V uses mip/PLIC/CLINT and mret instruction.

## 9. Non-Maskable Interrupt (NMI)

Cannot be disabled and is used for catastrophic failures such as memory corruption and critical hardware faults. NMI bypasses normal interrupt enable logic.

## Quick Revision Sheet

mip → Pending interrupt

mie → Which interrupts are enabled

mstatus → Processor status + MIE

mtvec → ISR address

mepc → Return address

mcause → Cause of interrupt

mret → Return from interrupt